

Πανεπιστήμιο Πειραιώς

Τμήμα Πληροφορικής

ΠΜΣ "Προηγμένα Συστήματα Πληροφορικής - Ανάπτυξη Λογισμικού και Τεχνητής Νοημοσύνης"

Μάθημα: Προηγμένη Βαθιά Μάθηση με Python

Θέμα Εργασίας:

Set-Based Learning with Emergent Label Dynamics: Theoretical Foundations and Implementation Using Permutation-Invariant Models

Ονοματεπώνυμο: **Παναγιώτης Μακρής**

Αριθμός Μητρώου: **24023** ΤΝΕΛ

Email: Pmakris.K@hotmail.com

Περιεχόμενα

1. Εισαγωγή & Διατύπωση Προβλήματος 2

2. Theoretical Background of Architectures 3

- **Deep Sets** 3
- **Set Transformer** 3
- **Janossy Pooling** 4
- **PointNet++** 4

3. Data Engineering & Dynamic Training Pipeline 4

4. Experimental Results 6

4.1 Citeseer Node Classification 6

4.2 Synthetic Dataset (Synthetic Task) 8

5. Conclusions & Future Extensions 11

6. Tool Usage Reflection 12

7. References 14

1. Εισαγωγή & Διατύπωση Προβλήματος

This assignment studies the ability of neural architectures to draw conclusions in classification problems, where the label of each point does not depend only on its own features, but is directly connected to the structure of its local neighborhood. To solve these problems, the models must be able to process sets without being affected by the input order of the data, showing permutation invariance.

A main innovation of our approach is the creation of a "data-agnostic" code (independent of the data format), which is not strictly tied to specific datasets. Instead, the training pipeline was designed to adapt dynamically and not be strictly limited to the initial requirements of the assignment. More specifically, the study is conducted on the following datasets:

- **Synthetic Dataset**: An artificial binary classification dataset, where the label of a point is flipped when the average of its neighbors creates ambiguity. It is worth noting that we have added the option to fully customize this dataset, allowing the free adjustment of the number of generated samples and the number of dimensions, depending on the needs of the experiment.
- **Citeseer Node Classification:** A real graph (citation graph), where each node is described by a 3,703-dimensional vector and is classified into six different document categories.
- **Custom Datasets**: Thanks to the flexibility of our implementation, the direct import of any custom data is supported. The program automatically recognizes the input dimensions and the number of classes of each new dataset, ensuring that the model training and the calculation of the metrics will be executed correctly in every new use.

2. Theoretical Background of Architectures

In this assignment, four different deep learning architectures were implemented from scratch:

- Deep Sets: It is the basic approach for processing sets. A network extracts features from each element independently, they are added together to create invariance, and another network produces the final prediction.

- Set Transformer: It extends the logic of Deep Sets by adding attention mechanisms. Through the Set Attention Block (SAB), it models the interactions between the elements of the neighborhood, while the Pooling by Multihead Attention (PMA) summarizes the information. In our implementation, a custom MultiHeadAttention mechanism was built.

- **Janossy Pooling:** It calculates functions by finding the average of the predictions over different permutations of the input set. In our practical implementation, a recurrent neural network (LSTM) was used to read random permutations of the neighborhood.
- **PointNet++:** An architecture for hierarchical feature extraction, originally designed for point clouds. For our problem, it was adapted to find local structures by calculating distances and extracting features from the k-nearest neighbors.

3. Data Engineering & Dynamic Training Pipeline

The training process was designed with a focus on the optimal mathematical navigation on the cost surface of each architecture separately. The key point of the system is finding an ideal Learning Rate right from the start, so that convergence is achieved smoothly and steadily, making aggressive corrections unnecessary during training. This way, the models adapt quickly to different datasets.

More specifically, the methodology followed includes:

- **Cost Surface Mapping (LR Range Test):**

  Instead of an arbitrary or empirical choice of the step size, the system tracks the loss function and finds the point of maximum negative slope, which is the limit speed just before the function starts to diverge.

- **Mathematical Smoothing (Scaling Factor):**

  Using the limit speed would cause huge jumps in the weight space, with the risk of overshooting the local minimum. To ensure smooth convergence, the detected maximum LR is divided by a strict smoothing factor. This adjustment guarantees that each model starts with its own step, able to cross the cost surface smoothly, without over-activating the neurons.

- **Organic Convergence and Early Stopping:**

  Because of the optimal initial choice of the learning rate, the error curve naturally reaches the plateau of the local minimum. Since the approach to the minimum is done with controlled steps, oscillation is avoided. When normal stagnation happens (bias-variance balance), the Early Stopping mechanism precisely stops the training. This prevents the network from simply memorizing the data noise (overfitting) and keeps the best weights for generalization.

- **Lazy Loading vs. Out-of-Memory Errors**:

  In the initial handling of the Citeseer dataset, pre-allocating memory for the neighborhood matrices (3312 nodes × 100 neighbors × 3703 features) caused the RAM to run out. The problem was fixed by redesigning the CiteseerDataset class, which now loads the data dynamically (Lazy Loading). It only builds the neighborhood matrix at the exact moment each batch is accessed.

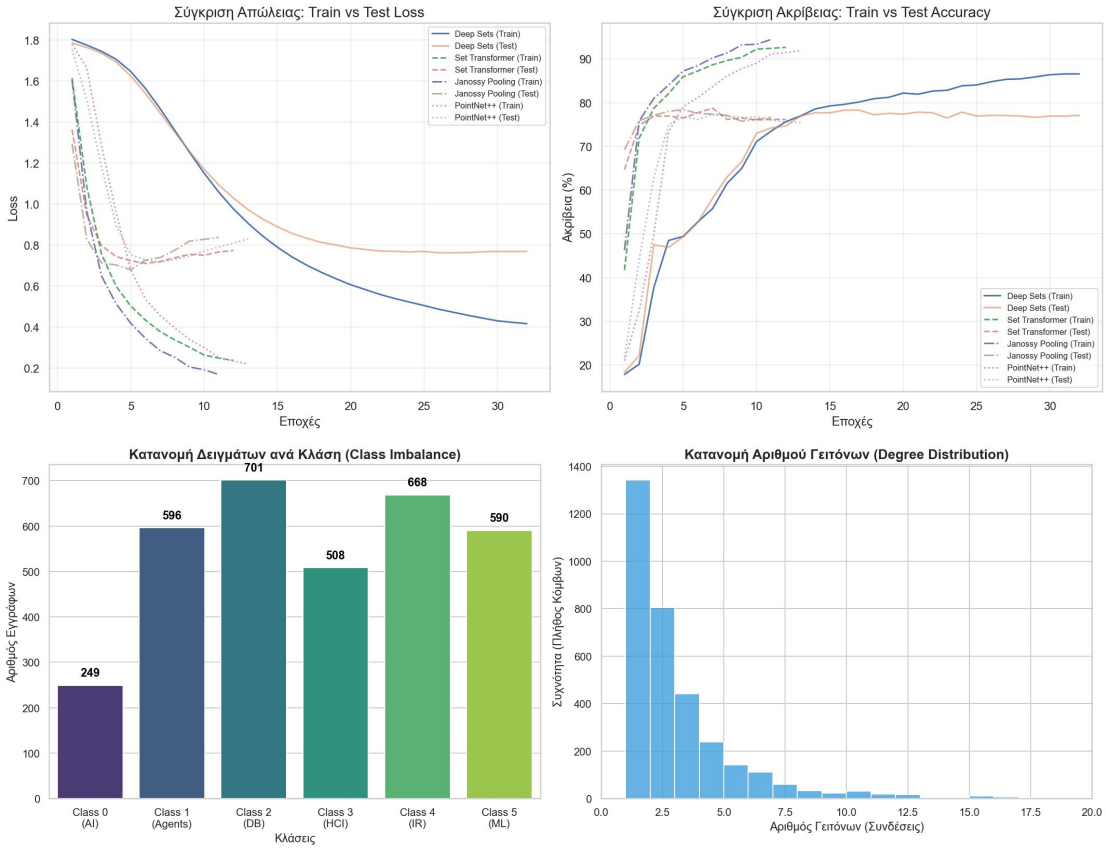
- **Flexibility & Dynamic Adaptability:**
  The system was built using Software Engineering principles, dynamically extracting the input dimensions (features) and the number of classes. Through the MyCustomDataset template class, the pipeline can take in new data without needing to change anything in the neural network architectures.

4. Experimental Results

4.1 Citeseer Node Classification

The models were trained using the Early Stopping mechanism (with a patience of 6 epochs). The final results on the Test Set are summarized in the table below:

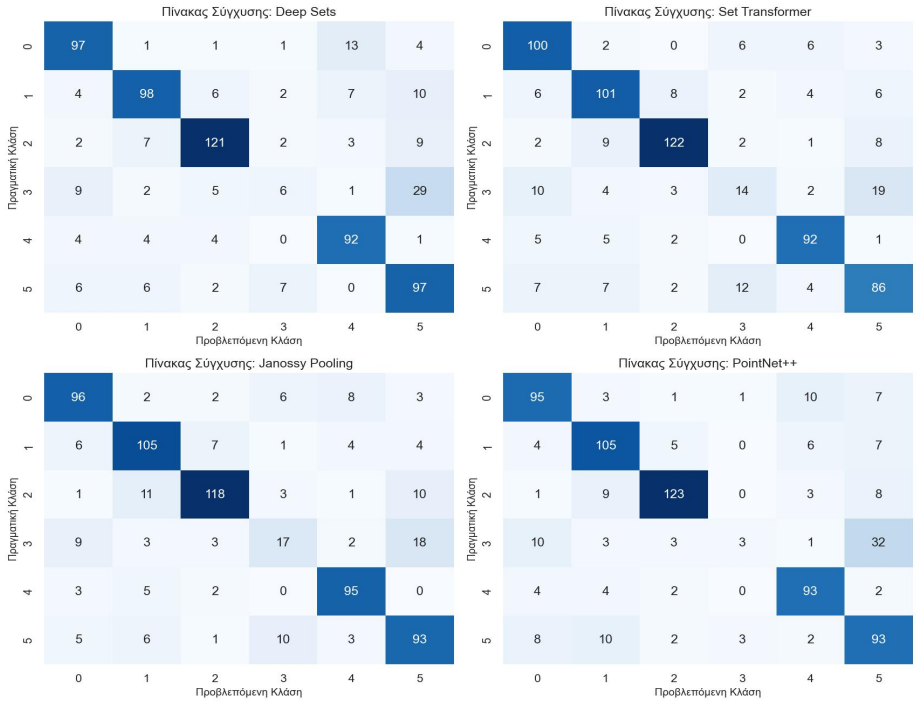
MODEL	Test Accuracy	Train Accuracy
Janossy Pooling	79.03%	87.20%
Set Transformer	77.68%	87.24%
PointNet++	77.22%	83.62%
Deep Sets	77.07%	84.79%



As we can see in the training charts, Janossy Pooling achieved the highest accuracy (79.03%) on the Test Set. At the same time, the other models, although they reached high accuracy levels during training (87.24%), showed a slightly larger gap between Train and Test Accuracy, which indicates a stronger tendency to overfit.

It is worth noting that the Training Accuracy naturally tends to increase constantly over the epochs, now reaching 87-88%. However, our final evaluation is strictly based on the performance on the Test Set. The continuing rise of accuracy on the training set—while at the same time the Test Loss becomes stable, as clearly seen where the curves split (diverge)—shows that the network is starting to simply memorize the data noise. At this point, we see how critical Early Stopping is, as it stopped the process right on time. This way, the system "locked" the weights exactly at the moment when the ability to generalize was highest, preventing further overfitting.

Finally, from analyzing the Confusion Matrices, we found that all models had a lot of trouble predicting Class 3 correctly, often classifying it wrongly as Class 5 or Class 0.



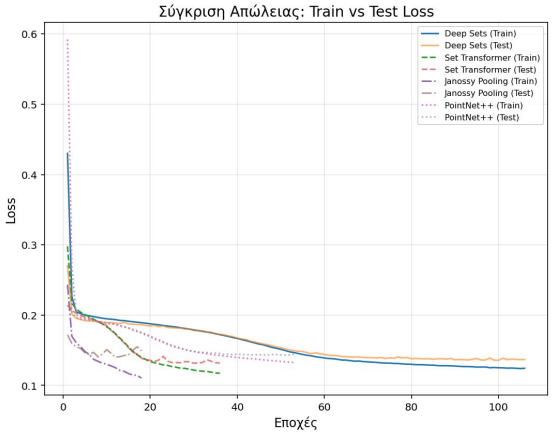
Given that this class has enough samples, this issue is not caused by class imbalance. Instead, it indicates a possible lack of homophily in the graph. This makes the local neighborhood features less clear, causing the models to get confused when they aggregate the information.

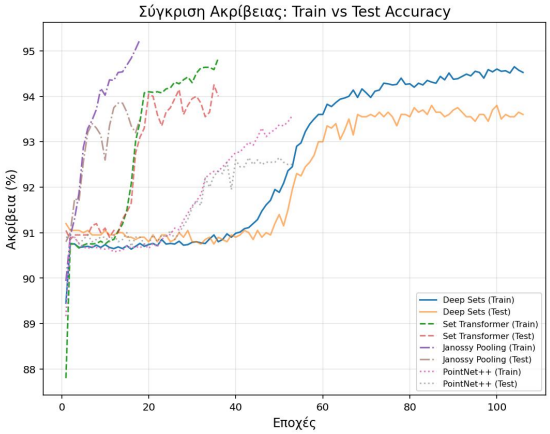
4.2 Synthetic Dataset (Synthetic Task)

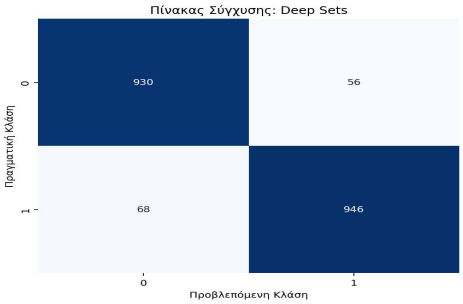
To evaluate how the models behave under controlled conditions of local interaction, the system was set up to dynamically generate a synthetic dataset of 10,000 samples in $d = 16$ dimensions with 2 output classes. In this space, the natural overlap of the data caused 11.1% of the labels to flip, based on the strict neighborhood rules. This added a realistic and measurable level of mathematical noise.

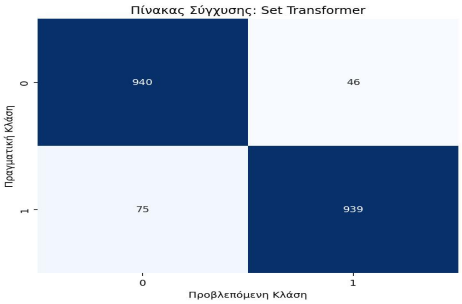
The final results are summarized in the table below:

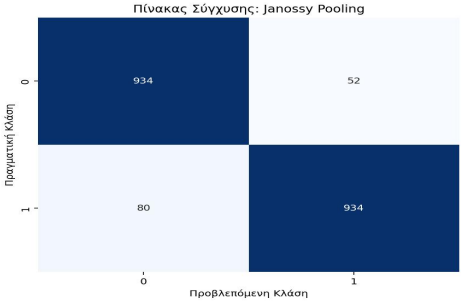
MODEL	Test Accuracy	Train Accuracy
Set Transformer	93.95%	94.30%
Deep Sets	93.80%	94.60%
Janossy Pooling	93.40%	94.35%
PointNet++	92.55%	93.11%

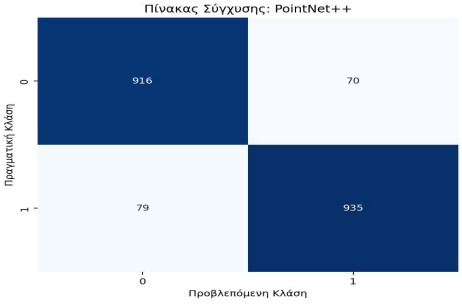












As seen in the training charts (see Figure image), the optimization process shows that the Set Models adapt extremely well to the topological noise. The models do not reach a fake 100%, but they become stable in the range of 92.5% - 94.0%. This gets impressively close to the best theoretical limit (Bayes Error Rate), which is caused by the 11.1% intentional overlap.
Special mention must be made of the Early Stopping mechanism, which stopped the training right on time (as seen by the early stop of curves like Janossy Pooling at epoch 18). This prevented the networks from overfitting and simply memorizing the flipped samples.

The ability to generalize correctly is also proven by the Confusion Matrices (see image). The classification errors are split perfectly equally between the two classes. This proves that the networks learned the real rule for dividing the space, without creating a statistical bias for any specific category.

Discussion: The Curse of Dimensionality

During the investigation of the synthetic dataset in a test environment of extremely high dimensionality ($d = 128$), a complete lack of overlaps was observed (0% flipped labels), with the models immediately achieving 100% accuracy. This phenomenon is the practical proof of the "Curse of Dimensionality": as the dimensions of the space increase, the Euclidean distances between the points grow exponentially, making the local neighborhoods completely homogeneous and moving the class borders far apart.

As a **Future Extension** of this study, we suggest adding an adaptive mechanism (adaptive ϵ-threshold) during the creation of the dataset. This mechanism will dynamically adjust the ambiguity

limit based on the standard deviation of the average neighbor labels (instead of a fixed value), ensuring that a realistic level of mathematical complexity is kept, no matter the number of dimensions.

5. Conclusions & Future Extensions

The evaluation of the Deep Sets, Set Transformer, Janossy Pooling, and PointNet++ architectures confirms that the ability to represent sets depends mostly on the basic topological features of the underlying graph, rather than the absolute capacity of each network. In environments with controlled noise, the models consistently stop at the theoretical Bayes limit, proving their mathematical stability. On the other hand, applying them to real data showed that the structural complexity—and especially the lack of homophily, as seen in Citeseer—sets the real upper limit for classification accuracy, limiting the separation power of local sums. Regarding the management of this complexity, Janossy Pooling showed the greatest strength in generalization, although it trades this strength for a heavy computational load due to its sequential nature.

For the future extension of the research and the improvement of the training pipeline, the following targeted interventions are suggested:

- **Dynamic LR Scheduling**: Even though using a fixed step through the LR Range Test ensured a smooth initial convergence, adding a dynamic reduction mechanism is necessary. Shrinking the step dynamically in the final stages of training will allow the model to navigate deeper into the local minimum of the cost function, maximizing the final accuracy before Early Stopping is activated.

- **Adaptive ϵ-Thresholding:** To deal with the "Curse of Dimensionality" in high-complexity spaces (e.g., $\mathbb{R}^{128}$), it is suggested to replace the fixed parameter ϵ with a dynamic function. Adjusting the threshold based on the standard deviation of the average neighbor labels will make sure that the noise is kept at realistic levels, preventing the fake and absolute separation of the classes.

- **REST API Development (Model-as-a-Service):** Taking advantage of the flexibility of the MyCustomDataset class, which already handles any data format dynamically, it is suggested to integrate the final pipeline into a REST API. This will allow external web applications to feed the system with new datasets and immediately receive the classification predictions, turning the model from a local experiment into a fully independent service.

6. Tool Usage Reflection

In this assignment, Large Language Models (LLMs) were used carefully and strictly as advisory tools, following the rules of academic ethics and the allowed ways of use. The role of the tools was limited to conceptual discussion, code optimization, and debugging.

What queries did you make?

- **Debugging & Architecture Queries:** "How can I handle the Out-Of-Memory (OOM) error when building dense neighborhood matrices in Citeseer using PyTorch?"

- **Conceptual Exploration Queries:** "What are the practical advantages of a fixed optimal LR (through LR Range Test) compared to dynamic approaches (like ReduceLROnPlateau) when training Permutation-Invariant networks?"

- **Theoretical Analysis Queries:** "Exactly how does the concept of Euclidean distance change in the synthetic data when we increase the dimensions from $d = 16$ to $d = 128$, and how does this explain the zero flips in the labels? "

How did you use and verify the output?

No design suggestion was ever copied directly. For example, the tool's suggestion to avoid the OOM error led to our own, entirely custom design of the Lazy Loading process inside the CiteseerDataset class, which was practically verified by checking the huge drop in RAM usage during runtime. Similarly, the theoretical discussions about the Bayes Error Rate were directly verified by our own experimental metrics: by watching the Early Stopping mechanism end the training at 93.8% (in $d = 16$ with 11.1% noise), we practically confirmed the theoretical assumption of statistical generalization.

In addition, the language models were used as academic proofreading tools. Starting from the strict mathematical phrasing of the conclusions, alternative wordings were asked in order to connect pure mathematical thought with the standard terminology of deep learning. The core of the reasoning, the creation of the arguments, and the analysis of the metrics are exclusively the product of personal mental work, with the tool helping strictly to improve the flow and grammar of the final text, following the standards of a professional technical report.

What did you learn?

The advisory use of the tool deepened our understanding of the relationship between mathematical topology and neural architectures. We learned to read the flat part of the training curve not as the model performing poorly, but as successfully recognizing the built-in noise of the data. Additionally, we understood the "Curse of Dimensionality" deeper on a practical level, realizing how the vast nature of high dimensions destroys the local ambiguity, making dynamic threshold functions (adaptive ϵ) necessary.

7. References

1. Guttag, J. V. (2015). *Υπολογισμοί και προγραμματισμός με Python*. (Ελληνική Έκδοση).
2. Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., & Teh, Y. W. (2019). Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning* (pp. 3744-3753). PMLR.
3. Murphy, R. L., Srinivasan, B., Rao, V., & Ribeiro, B. (2018). Janossy pooling: Learning deep permutation-invariant functions for variable-size inputs. *arXiv preprint arXiv:1811.01900*.
4. Qi, C. R., Yi, L., Su, H., & Guibas, L. J. (2017). Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *Advances in neural information processing systems*, 30.
5. Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., & Smola, A. J. (2017). Deep sets. *Advances in neural information processing systems*, 30.